

European Championship in Trustworthy AI

Supervisors: Adam Dziedzic and Franziska Boenisch

Title: Text Watermark Localization

Developed by: Maitri Shah and Louis Kerner



Goal

Participants are given documents containing a mixture of watermarked and non-watermarked text. For every token in each text document, you must predict a confidence score indicating whether that token was generated while a watermarking process was active.

This is different from asking whether a token matches a watermark detection rule. Many clean tokens satisfy a detection rule by chance, and some genuinely watermarked tokens fail to show a strong detector signal. Ground truth labels come from tracking when watermark application was switched on during generation, not from running a detector after the fact.

The objective is to localize watermarked text regions as accurately as possible while maintaining a very low false-positive rate.

Each token belongs to one of two evaluation classes:

- **0**: non-watermarked
- **1**: watermarked

Watermarked tokens may have been generated using any of the watermarking methods represented in the dataset.

Participants are not required to identify the watermark family. The task is token-level localization of watermark-active regions.

Challenge

The dataset contains text generated using the following watermarking schemes: TextSeal, Gumbel-Max, Unigram, and KGW. Each document is one tokenized continuation generated with a possible mixture of clean generated text, human-written text, and one or more spans where a watermarking process was active, possibly using different schemes across spans.

Specification

Participants are provided with a YAML file containing the exact tokenizer, watermark keys, and detector parameters used in the task. The YAML also specifies the official watermark repositories and the exact commits used, so participants can download these repositories to access the corresponding detector implementations. However, participants are not told which watermarking scheme is active at a particular token or span, and detector outputs alone are not sufficient: a token can satisfy a detection rule by chance without being part of an active watermarking process, and a token generated under an active process can fail to

satisfy the rule. The goal is to combine or improve the available detection signals to localize watermark-active regions, not to reproduce raw detector scores.

A single document may contain:

- clean model-generated regions,
- human-written regions,
- one or more watermarked regions,
- multiple watermarking methods in a document,
- clean gaps between watermarked spans.

A successful system must therefore:

- detect several different watermarking signals,
- distinguish watermarked tokens from nearby clean tokens,
- identify boundaries between regions,
- avoid incorrectly flagging human or clean text,
- produce meaningful token-level confidence scores.

Evaluation Metric

Submissions are evaluated using True Positive Rate (TPR) at 0.1% False Positive Rate (FPR).

$$TPR@0.1\%FPR$$

Your task is to assign a continuous confidence score between 0 and 1 for every token in the test data, indicating how likely it is that the token is watermarked.

Scores ≥ 0.5 $\rightarrow$ watermarked

Scores < 0.5 $\rightarrow$ clean (non-watermarked)

All tokens belonging to documents in the relevant leaderboard subset are pooled before the metric is calculated.

Higher scores indicate better watermark localization.

Dataset

The complete dataset contains 1,500 documents divided into:

Training set: 90 documents

Validation set: 90 documents

Test set: 1,320 documents

Documents have variable lengths and may contain several different text-generation regions. The training and validation sets include binary token-level labels. The test set does not include labels.

The dataset contains TextSeal, Gumbel-Max, Unigram and KGW watermarks.

Dataset is available at: https://huggingface.co/datasets/SprintML/watermark_localization

[hackathon_setup.sh](#) provided to you creates your team folder, downloads the datasets and creates a per task environment for you.

Training and validation records

Each training or validation record contains the following fields:

```
{
  "document_id": "train_1",
  "text": "Paul's statement",
  "token_ids": [25300, 594, 5114],
  "token_pieces": ["Paul", "'s", " statement"],
  "labels": [0, 0, 1]
}
```

The fields are:

- `document_id`: unique identifier for the document.
- `text`: readable decoded text.
- `token_ids`: the authoritative tokenizer output for the document.
- `token_pieces`: readable representations of the individual tokenizer pieces.
- `labels`: binary token-level ground truth.

The arrays `token_ids`, `token_pieces`, and `labels` have the same length.

The token at position `i` is represented by:

`token_ids[i]`

`token_pieces[i]`

`labels[i]`

Test records

Each test record contains:

```
{
  "document_id": "1",
  "text": "The complete document text...",
  "token_ids": [25300, 594, 5114],
  "token_pieces": ["Paul", "'s", "statement"]
}
```

The labels are hidden. Participants must produce exactly one confidence score for every value in `token_ids`.

Tokenization

The dataset uses the tokenizer associated with: Qwen/Qwen2.5-7B-Instruct.

You should treat `token_ids` as the authoritative token sequence. The readable `text` field

should not be retokenized in order to determine prediction length, because decoding and retokenization are not guaranteed to reproduce exactly the same token sequence. The `token_pieces` field is provided to make token boundaries easier to inspect.

All tokens in each released document, including chat-like markers and apparent user or assistant messages, are included in the evaluation and require a prediction.

Some tokenizer symbols may look unfamiliar. For example:

- `Ġword` indicates a token with a preceding space.
- `Ġ` indicates a newline.
- a single word may be divided into several token pieces.

These symbols are normal tokenizer representations.

Additional Resources

`task_template.py` is provided to help participants begin quickly. It includes:

- Code to unzip and load the dataset
- Example code to generate, validate and save continuous predictions for submission
- Participants can modify or extend the provided code with custom architectures, preprocessing, or features.

You are supposed to submit your scores using the code file `submission_template.py` provided. Remember to replace `YOUR_API_KEY_HERE` with your actual API-key (keep the double quotes), and replace `/PATH/FILE.jsonl` with your real results. Submissions must follow the required file format described below.

Important KGW Note. KGW greenlists were generated using `torch.randperm` on a CUDA generator (Philox). Recomputing greenlists on CPU does not reproduce the dataset signals and yields effectively random greenlist assignments, causing roughly one third of KGW-watermarked tokens to lose their watermark signal.

Submission Format

The submission `.jsonl` file must contain one JSON object per test document. Each object must contain exactly `document_id` and `scores`

Example:

```
{"document_id": "1", "scores": [0.02, 0.04, 0.91, 0.87]}
{"document_id": "2", "scores": [0.12, 0.16, 0.21]}
```

The score at position `i` is the predicted confidence that `token_ids[i]` is watermarked.

Submission Requirements

- Every test document must appear exactly once.
- Every expected `document_id` must be present.
- No additional document IDs are allowed.
- Duplicate document IDs are not allowed.

- Every row must be a valid JSON object.
- Each row must contain exactly the document_id and scores.
- Scores must be a one-dimensional list.
- Each score must be numeric and finite.
- Every score must lie within [0, 1].
- The number of scores must exactly equal the number of test tokens in that document.
- Participants must not submit hard labels, text, token IDs, token pieces, or watermark-family predictions.

Invalid submissions are rejected automatically.

Scoring

All valid submissions are evaluated against hidden token-level ground truth. The test documents are divided deterministically into two leaderboard subsets using a hash of document_id.

- Public leaderboard: 30% of test documents
- Private leaderboard: 70% of test documents

All tokens from a document remain in the same subset. The public leaderboard score is shown during the competition.

Leaderboard

After evaluation, your results can be found in the leaderboard:

- You can access the leaderboard for this task at https://35.192.205.84/leaderboard_page. This will help you to compare your solutions with other teams and see where you stand.
- The leaderboard shows the best result per team only. As output to your request, you will get back the score for your current submission. If it is lower than the score saved in the leaderboard, the score will not be updated.

References

1. “A Watermark for Large Language Models” John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, Tom Goldstein
<https://proceedings.mlr.press/v202/kirchenbauer23a.html> [ICML 2023]
2. “TextSeal: A Localized LLM Watermark for Provenance & Distillation Protection” Tom Sander, Hongyan Chang, Sylvestre-Alvise Rebuffi, Tomáš Souček, Tuan Tran, Valeriu Lacatusu, Alexandre Mourachko, Surya Parimi, Christophe Ropers, Rashel Moritz, Vanessa Stark, Hady Elsahar, Pierre Fernandez
<https://arxiv.org/pdf/2605.12456> [arXiv 2026]
3. “Watermarking GPT Outputs” (Gumbel-Max) Scott Aaronson and Hendrik Kirchner. (2023). [AK’23] <https://www.scottaaronson.com/talks/watermark.ppt> ; <https://scottaaronson.blog/?m=202302>
4. “Provable Robust Watermarking for AI-Generated Text” Xuandong Zhao, Prabhanjan Ananth, Lei Li, Yu-Xiang Wang <https://openreview.net/forum?id=SsmT8aO45L> [ICLR 2024]